

人工智能与机器学习

2025年9月

4.1 标量、向量、矩阵和张量

2

标量：物理学中，标量（或作纯量）指在坐标变换下保持不变的物理量。用通俗的说法，标量是只有大小，没有方向的量。

向量：在数学中，向量指具有大小和方向的量。它可以形象化地表示为带箭头的线段。箭头所指：代表向量的方向；线段长度：代表向量的大小。

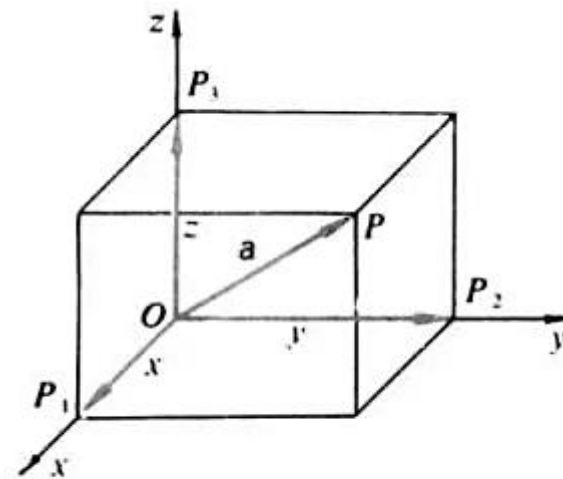


图 2 向量的坐标表示

矩阵：在数学中，矩阵（Matrix）是一个按照长方阵列排列的复数或实数集合

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ a_{31} & a_{32} & \cdots & a_{3n} \\ \cdots & \cdots & & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

4.1 标量、向量、矩阵和张量

2

张量 (Tensor) 是深度学习 中一个非常重要的概念，它是一种多维数组，用于表示和处理数据。张量可以看作是标量、向量和矩阵的推广，是深度学习框架（如TensorFlow、PyTorch）中的核心数据结构。

张量指的是N轴 (axis) 的数据。轴的个数也被称为**阶** (order, 也叫秩) 。

标量 (Scalar) : **0 阶的张量**，只有一个数字

向量 (Vector) : **1阶的张量**，也叫矢量，一个数组

矩阵 (Matrix) : **2阶的张量**，2个轴通常称为行和列

4.1 标量、向量、矩阵和张量

2

张量是一个多维数组，其维度（或阶数）表示数组的“深度”。具体来说：

- **0阶张量**：标量（Scalar），只有一个数值，例如 5。
- **1阶张量**：向量（Vector），一维数组，例如 $[1, 2, 3]$ 。
- **2阶张量**：矩阵（Matrix），二维数组，例如：

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

- **3阶张量**：三维数组，例如一个形状为 $2 \times 3 \times 4$ 的数组。
- **更高阶张量**：可以有任意多个维度，例如 $4 \times 3 \times 2 \times 2$ 的四阶张量。

4.1 标量、向量、矩阵和张量

2

通俗易懂的理解：把张量的“阶”想象成“数据的索引个数”或者“需要多少个坐标才能定位一个数据点”。

核心思想：张量的“阶” = 需要的索引数量

阶数越高，你需要的信息“地址”就越详细。

第0阶：标量 (Scalar)

- 通俗比喻：一个孤零零的点。比如：温度、体重、一个数字。
- 需要几个索引？ 0个。因为它就是一个单独的值，没有其他元素，所以不需要索引。它就像地图上的一个孤零零的灯塔，你只需要知道“它”在那里就行了。
- 深度学习中的例子：一个模型的损失值 (Loss)、一个神经元的偏置 (Bias)。

第1阶：向量 (Vector)

- 通俗比喻：一条笔直的线或一个清单。比如：一条水果清单 [苹果, 香蕉, 橙子]。
- 需要几个索引？ 1个。要找到清单里的“香蕉”，你只需要问一个问题：“第几个？” (比如，第2个)。它只有一个维度。
- 深度学习中的例子：一个单独的词通过嵌入层后得到的词向量 (Word Vector)、一个全连接层的输入或输出。

4.1 标量、向量、矩阵和张量

2

通俗易懂的理解：把张量的“阶”想象成“数据的索引个数”或者“需要多少个坐标才能定位一个数据点”。

第2阶：矩阵 (Matrix)

- 通俗比喻：一张表格或一个棋盘。比如：一张Excel表，一个水果货架。
- 需要几个索引？ 2个。要找到表格里的某个值，比如“小李的数学成绩”，你需要问两个问题：“第几行？”（小李是第3行）和“第几列？”（数学是第2列）。它有两个维度：行和列。
- 深度学习中的例子：一张灰度图片（宽度 x 高度）、一个全连接层的所有权重参数。

第3阶：立方体 (Cube)

通俗比喻：图书馆某一层房间，里面有很多书架，单个书架有多排，每排有很多本书。

需要几个索引？ 3个。要找到某一本书，你需要三个信息：第几个书架，第几排，第几列？

4.1 标量、向量、矩阵和张量

3

- 维 (dimension) 通常用来描述向量, 比如N维向量, 比如“128维特征值”。
- 这里的维和张量的轴定义不一样, 这种N维向量描述的仍然是向量 (即1阶张量), 其中的N描述的是数组的元素个数。
- 但是由于2阶张量 (即矩阵) 形似2维平面, 3阶张量形似3维空间, 而N阶张量又可以用N维数组来表示, 导致维有时会被拿来形容张量的阶。
- 为了避免混淆, 不要用“维”来描述张量的阶。

4.1 标量、向量、矩阵和张量

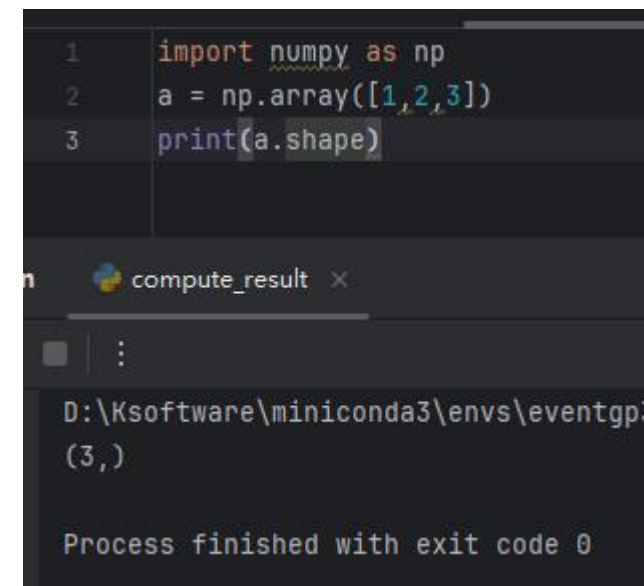
4

NumPy (Numerical Python) 是Python的一种开源的数值计算扩展。

这种工具可用于存储和处理大型矩阵，比Python自身的嵌套列表（nested list structure）结构要高效的多（该结构也可以用来表示矩阵（matrix）），支持大量的维度数组与矩阵运算，此外也针对数组运算提供大量的数学函数库NumPy提供了一个N维数组类型ndarray，它描述了相同类型的“items”的集合。

```
import numpy as np
a = np.array([1,2,3])
print(a.shape)
```

shape输出对应轴的维度



```
1 import numpy as np
2 a = np.array([1,2,3])
3 print(a.shape)
```

compute_result x

D:\Ksoftware\miniconda3\envs\eventgp3
(3,)

Process finished with exit code 0

4.1 标量、向量、矩阵和张量

5

- **标量** (scalar): 一个标量就是一个单独的数，它不同于线性代数中研究的其他大部分对象 (通常是多个数的数组)。我们用斜体表示标量。标量通常被赋予小写的变量名称。当我们介绍标量时，会明确它们是哪种类型的数。比如，在定义实数标量时，我们可能会说“令 $s \in \mathbb{R}$ 表示一条线的斜率”；在定义自然数标量时，我们可能会说“令 $n \in \mathbb{N}$ 表示元素的数目”。

```
import numpy as np
x = np.array(12)
y = np.array([12])
print(x,x.shape)
print(y,y.shape)
```

```
12 ()
[12] (1,)
```

x是标量

y是一维向量 (注意这里是向量不是张量)

4.1 标量、向量、矩阵和张量

5

```
compute_result.py x
1 import numpy as np
2 x = np.array(12)
3 y = np.array([12])
4 z = np.array([12, 12, 12])
5 print(x, "-----", x.shape)
6 print(y, "-----", y.shape)
7 print(z, "-----", z.shape)

n compute_result x
D:\Ksoftware\miniconda3\envs\eventgp38
12 ----- ()
[12] ----- (1,)
[12 12 12] ----- (3,)

Process finished with exit code 0
```

4.1 标量、向量、矩阵和张量

5

核心比喻：张量的shape就像"数据容器的尺寸标签"

想象你要描述一个仓库里存放东西的方式，shape就是告诉你这个仓库的**结构规格**。

从简单到复杂理解Shape

1. 标量 (0阶)：空盒子

- Shape: `()` (空括号)
- 比喻：一个盒子里只放一个苹果
- 理解：不需要任何维度信息，因为它就是一个单独的值

2. 向量 (1阶)：一行货架

- Shape: `(5,)`
- 比喻：一排5个格子的货架 [苹果, 香蕉, 橙子, 葡萄, 西瓜]
- 理解：只有一个数字 5，表示"这一排有5个格子"

3. 矩阵 (2阶)：一个货架

- Shape: `(3, 4)`
- 比喻：一个3行4列的货架

```
text

[
    [苹果, 香蕉, 橙子, 葡萄],
    [西瓜, 芒果, 桃子, 梨子],
    [柠檬, 樱桃, 菠萝, 椰子]
]
```

- 理解：(行数, 列数) = `(3, 4)`，表示"3行，每行4个格子"

4.1 标量、向量、矩阵和张量

5

核心比喻：张量的shape就像"数据容器的尺寸标签"，
想象你要描述一个仓库里存放东西的方式，shape就是告诉你这个仓库的结构规格。

4. 3维张量：一排货架

- **Shape:** (2, 3, 4)
- **比喻：** 仓库里有2排货架，每排有3层，每层有4个格子
- **理解：** (货架排数, 每排层数, 每层格子数)
- **真实例子：** 一张彩色图片的shape是 (高度, 宽度, 通道数)，比如 (224, 224, 3)

5. 4维张量：整个仓库

- **Shape:** (32, 224, 224, 3)
- **比喻：** 一个仓库有32个房间，每个房间有224个货架，每个货架的尺寸是224*3
- **理解：** (批次大小, 高度, 宽度, 通道数)
- **真实例子：** 深度学习中最常见的——一个批次的图片

4.1 标量、向量、矩阵和张量

5

tensor的shape与tensor的阶之间关系

看维度数：有几个数字就是几阶张量

$(5,)$ $\rightarrow$ 1阶张量（一个索引就能找到指定元素）

$(3,4)$ $\rightarrow$ 2阶张量（两个索引能找到指定元素）

$(2,3,4)$ $\rightarrow$ 3阶张量（三个索引能找到指定元素）

4.1 标量、向量、矩阵和张量

6

```
x = np.array([12, 3, 6, 14, 7])  
print(x.ndim)
```

ndim输出轴的数量

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

```
compute_result.py x  
1 import numpy as np  
2 x = np.array(12)  
3 y = np.array([12])  
4 z = np.array([12, 12, 12])  
5 print(x, "-----", x.shape, "-----", x.ndim)  
6 print(y, "-----", y.shape, "-----", y.ndim)  
7 print(z, "-----", z.shape, "-----", z.ndim)  
8  
n compute_result x  
:  
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\mywor  
12 ----- () ----- 0  
[12] ----- (1,) ----- 1  
[12 12 12] ----- (3,) ----- 1  
  
Process finished with exit code 0
```


4.1 标量、向量、矩阵和张量

7

- 矩阵 (matrix): 矩阵是一个二维数组, 其中的每一个元素被两个索引 (而非一个) 所确定。我们通常会赋予矩阵粗体的大写变量名称, 比如 $\mathbf{A}$ 。如果一个实数矩阵高度为 m , 宽度为 n , 那么我们说 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 。我们在表示矩阵中的元素时, 通常以不加粗的斜体形式使用其名称, 索引用逗号间隔。比如, $A_{1,1}$ 表示 $\mathbf{A}$ 左上的元素, $A_{m,n}$ 表示 $\mathbf{A}$ 右下的元素。我们通过用 “:” 表示水平坐标, 以表示垂直坐标 i 中的所有元素。比如, $\mathbf{A}_{i,:}$ 表示 $\mathbf{A}$ 中垂直坐标 i 上的一横排元素。这也被称为 $\mathbf{A}$ 的第 i 行 (row)。同样地, $\mathbf{A}_{:,i}$ 表示 $\mathbf{A}$ 的第 i 列 (column)。当我们需要明确表示矩阵中的元素时, 我们将它们写在用方括号括起来的数组中:

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{bmatrix}. \quad (2.2)$$

2.1 标量、向量、矩阵和张量

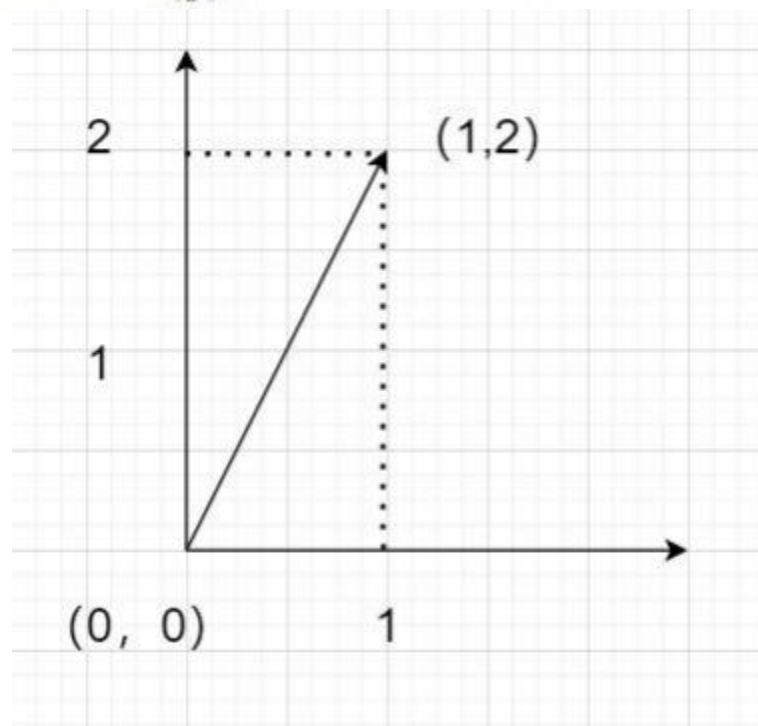
8

- 张量 (tensor): 在某些情况下, 我们会讨论坐标超过两维的数组。一般地, 一个数组中的元素分布在若干维坐标的规则网格中, 我们称之为张量。我们使用字体 **A** 来表示张量 “A”。张量 **A** 中坐标为 (i, j, k) 的元素记作 $A_{i,j,k}$ 。

此处的维指的是: 坐标轴的个数, 其实是二阶张量或三阶张量里面的阶

```
A= np.array([[1, 2, 3, 3], [4, 5, 6, 6], [7, 8, 9, 9]])  
A.shape = (3,4)
```

$A_{2,4}=6$



2.1 标量、向量、矩阵和张量

8

```
compute_result.py x
1 import numpy as np
2 A= np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
3
4 print(A)
5 print("#####")
6 print(A[2][3])
7 print("#####")
8 print(A.shape)
9

compute_result x
:
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
#####
12
#####
(3, 4)

Process finished with exit code 0
```

4.1 标量、向量、矩阵和张量

9

- 我们以numpy.ndarray类型的变量t的形状 (t.shape) 来说明:
 - (1,): 向量, 1阶张量, 只有1个数值
 - (3,): ?, ? 阶张量, ? 维向量, 有? 个数值
 - (5,): ?, ? 阶张量, ? 维向量, 有? 个数值
 - (2,3): ?, ? 阶张量, 有? 个数值, t[0]为? 维向量
 - (3,1): ?, ? 阶张量, 有? 个值, t[0]为? 维向量
 - (3,4,5): ? 阶张量, 有? 个数值, t[0,0]为? 维向量

(1,): 向量, 1阶张量, 只有1个数值

(3,): 向量, 1阶张量, 3维向量, 有3个数值

(5,): 向量, 1阶张量, 5维向量, 有5个数值

```
compute_result.py x
1 import numpy as np
2 x = np.array([6])
3 y = np.array([3, 6, 9])
4 z = np.array([1, 2, 3, 4, 5])
5 print(x, "-----", x.shape)
6 print(y, "-----", y.shape)
7 print(z, "-----", z.shape)

un compute_result x
D:\Ksoftware\miniconda3\envs\eventgp38\python
[6] ----- (1,)
[3 6 9] ----- (3,)
[1 2 3 4 5] ----- (5,)

Process finished with exit code 0
```

(2,3): 矩阵, 2阶张量, 有 6 个数值, t[0]为 3 维向量

(3,1): 矩阵, 2阶张量, 有 3 个值, t[0]为1维向量

```
compute_result.py x
8 m = np.array([[1,2,3],[4,5,6]])
9 n = np.array([[1],[2],[3]])
10
11 print(m)
12 print(m.shape, "~~~~~")
13 print("#####")
14 print(n)
15 print(n.shape)
16
```

```
n compute_result x
:
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe
[[1 2 3]
 [4 5 6]]
(2, 3) ~~~~~
#####
[[1]
 [2]
 [3]]
(3, 1)

Process finished with exit code 0
```

(3,4,5): 3 阶张量, 有60个数值, t[0,0]为 5 维向量

```
compute_result.py x
8
9 o = np.array([[[1,2,3, 4,5], [1,2,3, 4,5], [1,2,3, 4,5]
10
11 print(o)
12 print(o.shape)
13

n compute_result x
:
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks
[[[ 1  2  3  4  5]
 [ 1  2  3  4  5]
 [ 1  2  3  4  5]
 [ 1  2  3  4  5]]

 [[ 6  7  8  9 10]
 [ 6  7  8  9 10]
 [ 6  7  8  9 10]
 [ 6  7  8  9 10]]

 [[11 12 13 14 15]
 [11 12 13 14 15]
 [11 12 13 14 15]
 [11 12 13 14 15]]]
(3, 4, 5)

Process finished with exit code 0
```

4.1 标量、向量、矩阵和张量

10

张量的关键属性：

- 轴的个数（阶）

例如，3阶张量有3个轴，2阶张量（矩阵）有2个轴。这在 Numpy 等 Python 库中也叫张量的 ndim

- 形状（shape）

例如，矩阵示例的形状为 (3, 5)，3阶张量示例的形状为 (3, 3, 5)

这是一个整数元组，表示**张量沿每个轴的维度大小**

- 数据类型

（在 Python 库中通常叫作 dtype）。这是张量中所包含数据的类型，例如，张量的类型可以是 float32、uint8、float64 等。在极少数情况下是字符（char）张量。

*注意，Numpy（以及大多数其他库）中不存在字符串张量，因为张量存储在预先分配的连续内存段中，而字符串的长度是可变的，无法用这种方式存储。

4.1 标量、向量、矩阵和张量

11

```
x = np.array([[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]])
```

输出x.shape, x.ndim

```
x = np.array([[[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]]])
```

输出x.shape, x.ndim

4.1 标量、向量、矩阵和张量

11

```
x = np.array([[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]])
```

输出x.shape, x.ndim

```
compute_result.py x
1 import numpy as np
2 x = np.array([[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]])
3 print(x)
4 print(x.ndim)
5 print(x.shape)

compute_result x
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-as-GP\data_p
[[ 5 78  2 34  0]
 [ 6 79  3 35  1]
 [ 7 80  4 36  2]]
```

4.1 标量、向量、矩阵和张量

11

```
x = np.array([[[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]]])
```

输出x.shape, x.ndim

compute_result.py x

```
1 import numpy as np
2 x = np.array([[[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2,
3 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]]])
4
5 print(x)
6 print(x.ndim)
7 print(x.shape)
```

n compute_result x



D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-as-6P\data_pretreatment\compute_result.py

```
[[[ 5 78  2 34  0]
   [ 6 79  3 35  1]
   [ 7 80  4 36  2]]
```

```
[[ 5 78  2 34  0]
 [ 6 79  3 35  1]
 [ 7 80  4 36  2]]
```

```
[[ 5 78  2 34  0]
 [ 6 79  3 35  1]
 [ 7 80  4 36  2]]
```

Process finished with exit code 0

4.1 标量、向量、矩阵和张量

11

```
compute_result.py x
1 import numpy as np
2 A = np.array([[[1],[1], [1]]])
3 print(A)
4 print("#####")
5 print(A.shape)
```

compute_result x compute_result x

D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myw

```
[[[1]
 [1]
 [1]]]
#####
(1, 3, 1)
```

Process finished with exit code 0

```
compute_result.py x
1 import numpy as np
2 A = np.array([[[1],[1], [1]],[[1],[1], [1]]])
3 print(A)
4 print("#####")
5 print(A.shape)
```

compute_result x compute_result x

D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myw

```
[[[1]
 [1]
 [1]]
 [[1]
 [1]
 [1]]]
#####
(2, 3, 1)
```

Process finished with exit code 0

4.1 标量、向量、矩阵和张量

12

```
import torch
x = torch.tensor([[1, 2, 8, 9], [1, 2, 8, 9]]) # (2, 4)
y = torch.tensor([[5, 8, 8, 3], [5, 8, 8, 3]]) # (2, 4)

print(torch.cat((x, y), dim=0)) # 按行 (轴1) 叠加
print(torch.cat((x, y), dim=1)) # 按列 (轴2) 叠加

tensor([[1, 2, 8, 9],
        [1, 2, 8, 9],
        [5, 8, 8, 3],
        [5, 8, 8, 3]]) # 按行叠加 (4, 4)

tensor([[1, 2, 8, 9, 5, 8, 8, 3],
        [1, 2, 8, 9, 5, 8, 8, 3]]) # 按列叠加 (2, 8)
```

4.1 标量、向

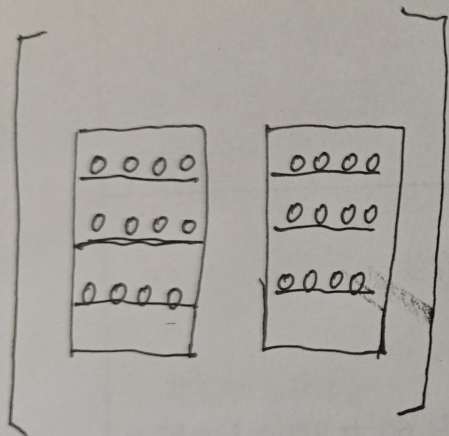
12

```
compute_result.py ×
1
2 import torch
3
4 x = torch.tensor([[1, 2, 8, 9], [1, 2, 8, 9]])
5 y = torch.tensor([[5, 8, 8, 3], [5, 8, 8, 3]])
6 print(x)
7 print(y)
8 print("#####")
9 print(torch.cat(tensors=(x,y),dim=0))
10 print(torch.cat(tensors=(x,y),dim=1))

compute_result ×
:
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\eve
tensor([[1, 2, 8, 9],
        [1, 2, 8, 9]])
tensor([[5, 8, 8, 3],
        [5, 8, 8, 3]])
#####
tensor([[1, 2, 8, 9],
        [1, 2, 8, 9],
        [5, 8, 8, 3],
        [5, 8, 8, 3]])
tensor([[1, 2, 8, 9, 5, 8, 8, 3],
        [1, 2, 8, 9, 5, 8, 8, 3]])

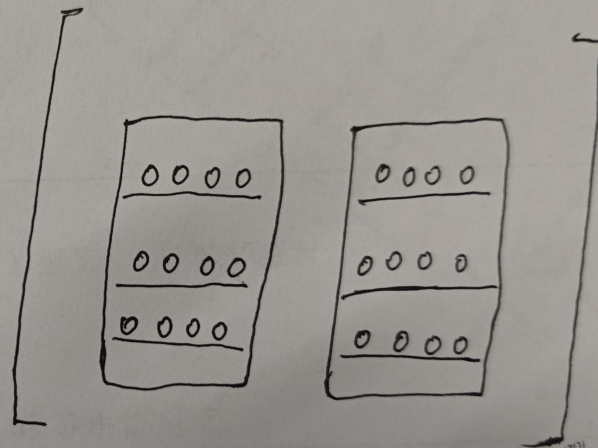
Process finished with exit code 0
```


x



2 x 3 x 4

y



2 x 3 x 4

```

1 import numpy as np
2 import torch
3
4 x = torch.tensor([[[1, 2, 3, 3], [1, 2, 4, 4], [1, 2, 5, 5]], [[1, 2, 6, 6], [1, 2, 7, 7], [1, 2, 8, 8]]])
5 y = torch.tensor([[[5, 8, 1, 1], [5, 8, 2, 2], [5, 8, 3, 3]], [[5, 8, 4, 4], [5, 8, 5, 5], [5, 8, 6, 6]]])
6 print(x)
7 print(y)
8
9 print("#####")
10 print(torch.cat( tensors: (x,y), dim=0))

```

in compute_result x

D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-as-GP\data_pretreatment\compute_result.p

```

tensor([[[1, 2, 3, 3],
          [1, 2, 4, 4],
          [1, 2, 5, 5]],
        [[1, 2, 6, 6],
          [1, 2, 7, 7],
          [1, 2, 8, 8]]])
tensor([[[5, 8, 1, 1],
          [5, 8, 2, 2],
          [5, 8, 3, 3]],
        [[5, 8, 4, 4],
          [5, 8, 5, 5],
          [5, 8, 6, 6]]])

```

#####

```

tensor([[[1, 2, 3, 3],
          [1, 2, 4, 4],
          [1, 2, 5, 5]],
        [[1, 2, 6, 6],
          [1, 2, 7, 7],
          [1, 2, 8, 8]],
        [[5, 8, 1, 1],
          [5, 8, 2, 2],
          [5, 8, 3, 3]],
        [[5, 8, 4, 4],
          [5, 8, 5, 5],
          [5, 8, 6, 6]]])

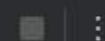
```

Process finished with exit code 0

```

1 import numpy as np
2 import torch
3
4 x = torch.tensor([[[1, 2, 3, 3], [1, 2, 4, 4], [1, 2, 5, 5]], [[1, 2, 6, 6], [1, 2, 7, 7], [1, 2, 8, 8]]])
5 y = torch.tensor([[[5, 8, 1, 1], [5, 8, 2, 2], [5, 8, 3, 3]], [[5, 8, 4, 4], [5, 8, 5, 5], [5, 8, 6, 6]]])
6 print(x)
7 print(y)
8
9 print("#####")
10 print(torch.cat( tensors: (x,y), dim=1))
11 # print(torch.cat((x,y), dim= 2))

```



D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-as-GP\data_pretreatment\compute_result.py

```

tensor([[[1, 2, 3, 3],
         [1, 2, 4, 4],
         [1, 2, 5, 5]],
        [[1, 2, 6, 6],
         [1, 2, 7, 7],
         [1, 2, 8, 8]]])
tensor([[[5, 8, 1, 1],
         [5, 8, 2, 2],
         [5, 8, 3, 3]],
        [[5, 8, 4, 4],
         [5, 8, 5, 5],
         [5, 8, 6, 6]]])

```

#####

```

tensor([[[1, 2, 3, 3],
         [1, 2, 4, 4],
         [1, 2, 5, 5],
         [5, 8, 1, 1],
         [5, 8, 2, 2],
         [5, 8, 3, 3]],
        [[1, 2, 6, 6],
         [1, 2, 7, 7],
         [1, 2, 8, 8],
         [5, 8, 4, 4],
         [5, 8, 5, 5],
         [5, 8, 6, 6]]])

```

compute_result.py x

```
1 import numpy as np
2 import torch
3
4 x = torch.tensor([[[1, 2, 3, 3], [1, 2, 4, 4], [1, 2, 5, 5]], [[1, 2, 6, 6], [1, 2, 7, 7], [1, 2, 8, 8]]])
5 y = torch.tensor([[[5, 8, 1, 1], [5, 8, 2, 2], [5, 8, 3, 3]], [[5, 8, 4, 4], [5, 8, 5, 5], [5, 8, 6, 6]]])
6 print(x)
7 print(y)
8
9 print("#####")
10 print(torch.cat(tensors=(x,y), dim=2))
```

compute_result x

D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-as-6P\data_pretreatment\compute_result.py

```
tensor([[[1, 2, 3, 3],
          [1, 2, 4, 4],
          [1, 2, 5, 5]],
        [[1, 2, 6, 6],
          [1, 2, 7, 7],
          [1, 2, 8, 8]]])
tensor([[[5, 8, 1, 1],
          [5, 8, 2, 2],
          [5, 8, 3, 3]],
        [[5, 8, 4, 4],
          [5, 8, 5, 5],
          [5, 8, 6, 6]]])
```

#####

```
tensor([[[1, 2, 3, 3, 5, 8, 1, 1],
          [1, 2, 4, 4, 5, 8, 2, 2],
          [1, 2, 5, 5, 5, 8, 3, 3]],
        [[1, 2, 6, 6, 5, 8, 4, 4],
          [1, 2, 7, 7, 5, 8, 5, 5],
          [1, 2, 8, 8, 5, 8, 6, 6]]])
```

Process finished with exit code 0

4.1 标量、向量、矩阵和张量

13

```
x = torch.tensor([1, 2, 8, 9])  
y = torch.tensor([5, 8, 8, 3])  
print(torch.cat((x, y), dim=0)) # 按行 (轴1) 相加 tensor([1, 2, 8, 9, 5, 8, 8, 3])  
print(torch.cat((x, y), dim=1)) # 按列 (轴2) 相加
```


4.1 标量、向量、矩阵和张量

13

compute_result.py x

```
1 import numpy as np
2 import torch
3
4 x = torch.tensor([1, 2, 8, 9])
5 y = torch.tensor([5, 8, 8, 3])
6 print(torch.cat(tensors=(x, y), dim=0))
7 print(torch.cat(tensors=(x, y), dim=1))
```

Run

compute_result x



```
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-as-GP\data_pretreatment\compute_result.py
tensor([1, 2, 8, 9, 5, 8, 8, 3])
Traceback (most recent call last):
  File "D:\myworks\event-as-GP\data_pretreatment\compute_result.py", line 7, in <module>
    print(torch.cat((x, y), dim=1))
IndexError: Dimension out of range (expected to be in range of [-1, 0], but got 1)

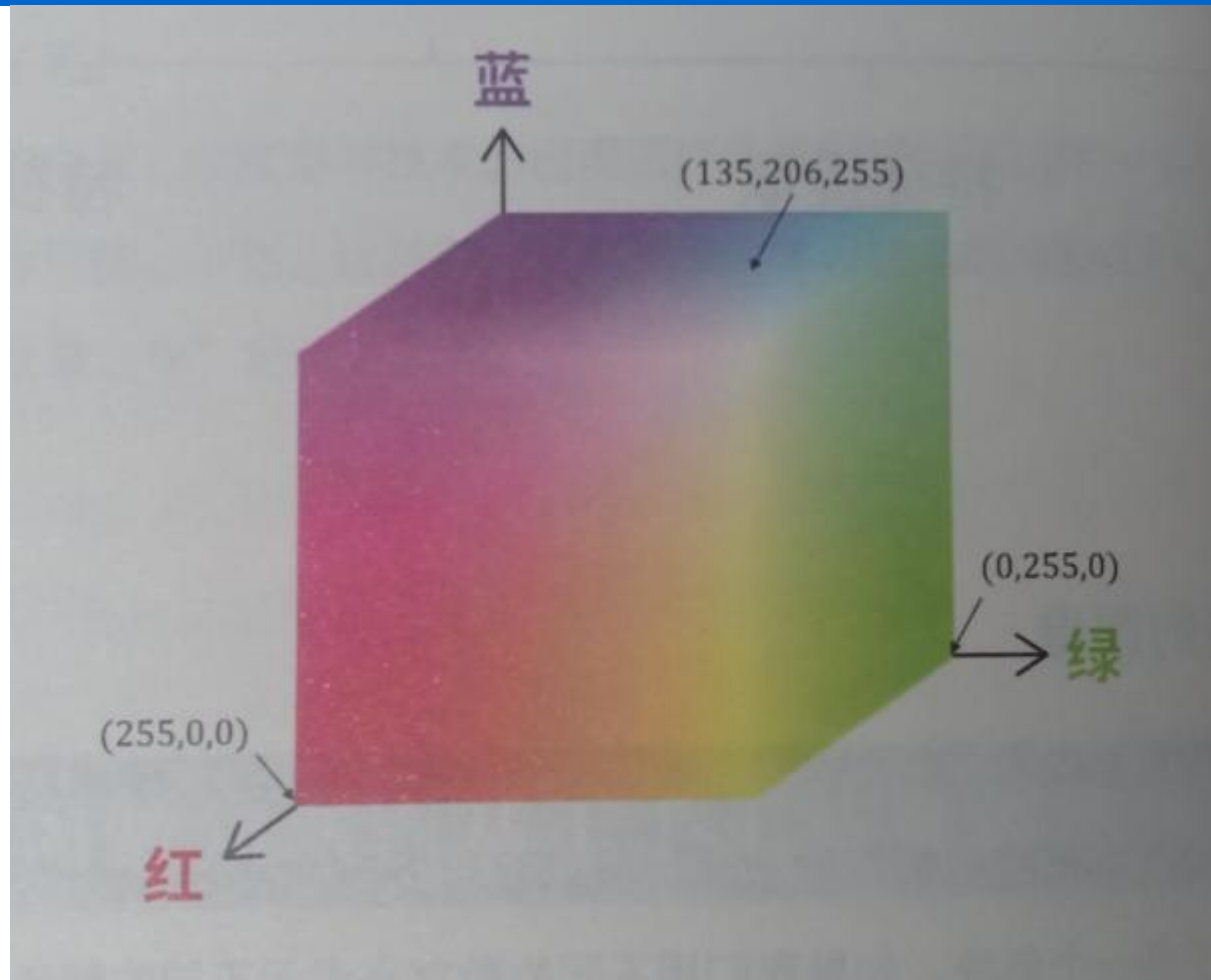
Process finished with exit code 1
```

4.1 标量、向量、矩阵和张量

14

channel: 三阶张量的高度

```
x = np.array([[[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]], [[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]]])
```



4.1 标量、向量、矩阵和张量

15

```
import numpy as np
from PIL import Image
x = np.array([[[5, 78, 2, 34, 0], [6, 79, 3, 35, 1], [7, 80, 4, 36, 2]],
x = np.transpose(x, (1, 2, 0))#将张量转换为 (H, W, C) 形式
x = x.astype(np.uint8) #转换为 uint8 数据类型
image = Image.fromarray(x)
image.show()
```



2.1 标量、向量、矩阵和张量

16

转置 (transpose) 是矩阵的重要操作之一。矩阵的转置是以对角线为轴的镜像, 这条从左上角到右下角的对角线被称为 **主对角线** (main diagonal)。图 2.1 显示了这个操作。我们将矩阵 $\mathbf{A}$ 的转置表示为 $\mathbf{A}^\top$, 定义如下

$$(\mathbf{A}^\top)_{i,j} = A_{j,i}. \quad (2.3)$$

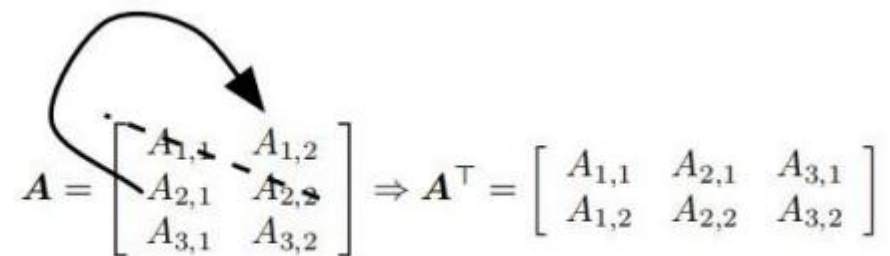

$$\mathbf{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} \Rightarrow \mathbf{A}^\top = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \end{bmatrix}$$

图 2.1: 矩阵的转置可以看成以主对角线为轴的一个镜像。

2.1 标量、向

16

```
1 import numpy as np
2 import torch
3
4 x = torch.tensor([[1,2],[3,4],[5,6]])
5 print(x)
6 z = torch.transpose(x, dim0: 0, dim1: 1)
7 print(z)
8 print("oooooooooooooooooooooooooooo")
9 y = torch.transpose(x, dim0: 1, dim1: 0)
10 print(x)
11 print(y)
12
```

Run compute_result x



```
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\eventgp38\compute_result.py
tensor([[1, 2],
        [3, 4],
        [5, 6]])
tensor([[1, 3, 5],
        [2, 4, 6]])
oooooooooooooooooooooooooooo
tensor([[1, 2],
        [3, 4],
        [5, 6]])
tensor([[1, 3, 5],
        [2, 4, 6]])
```

Process finished with exit code 0

2.1 标量、向量、矩阵和张量

17

向量可以看作只有一列的矩阵。对应地，向量的转置可以看作是只有一行的矩阵。有时，我们通过将向量元素作为行矩阵写在文本行中，然后使用转置操作将其变为标准的列向量，来定义一个向量，比如 $\mathbf{x} = [x_1, x_2, x_3]^\top$ 。

标量可以看作是只有一个元素的矩阵。因此，标量的转置等于它本身。

那么，张量如何转置？

2.1 标量、向量、矩阵和张量

17

```
compute_result.py x
17 # n = torch.transpose(m, 1,0)
18 # print(n)
19
20 x = torch.tensor(3)
21 print(x)
22 print(x.shape)
23 y = torch.transpose(x, dim0: 0, dim1: 1)
24 # z =torch.transpose(x, 1,0)

in compute_result x
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event-as-6P\data_pretreatment\c
tensor(3)
torch.Size([])
Traceback (most recent call last):
  File "D:\myworks\event-as-6P\data_pretreatment\compute_result.py", line 23, in <module>
    y = torch.transpose(x, 0, 1)
IndexError: Dimension out of range (expected to be in range of [-1, 0], but got 1)

Process finished with exit code 1
```

2.1 标量、向量、矩阵和张量

18

- **torch.transpose**(input, dim0, dim1, out=None)
- 函数返回输入矩阵input的转置。 **交换维度dim0和dim1**
- 参数:
 - input (Tensor) – 输入张量, 必填
 - dim0 (int) – 转置的第一维, 默认0, 可选
 - dim1 (int) – 转置的第二维, 默认1, 可选

核心思想：轴的交换

首先，忘掉“行列互换”这个二维概念。高维转置的核心是“轴”（Axis）的重新排列。

一个三阶张量有三个轴，我们通常称它们为：

- 轴0 (Axis 0)：通常代表“层”、“样本”或“通道”（取决于上下文）
- 轴1 (Axis 1)：通常代表“行”
- 轴2 (Axis 2)：通常代表“列”

一个三阶张量的形状是（深度，高度，宽度）或（层数，行数，列数）。

转置就是改变这些轴的顺序。

方法一：最直观的“书架”或“魔方”比喻

想象一个三阶张量是一个书架：

- 轴0 (深度)：第几个书架（第0个，第一个，...）
- 轴1 (高度)：一个书架有好几排书（从上至下）
- 轴2 (宽度)：每一排有好多本书，第几本书（从左至右）

1. 转置 (0, 1, 2) -> (0, 2, 1)

- 操作：保持书架不变，交换书架里的“行”和“列”。
- 效果：书架还是那个书架， 书架本身被旋转了90度。原来放在第2行第3列的书，现在跑到了第3行第2列。这类似于对书架的每一层单独做矩阵的转置。

上图：对后两个轴 (1,2) 进行转置，即 `a.transpose(1, 2)` 或 `np.transpose(a, (0, 2, 1))`

2. 转置 (0, 1, 2) -> (1, 0, 2)

- 操作：交换“格”和“行”。
- 效果：这相当于把书架拆了，然后用一种新的方式重新组装。原来的第几格，变成了现在的第几行。
 - 原来第0格的第1行书，现在变成了第1格的第0行书。
 - 想象一下：你把书架每一格的第1行书都抽出来，堆在一起形成新书架的第一格；然后把每一格的第2行书抽出来，堆成新书架的第二格，以此类推。

上图：对前两个轴 (0,1) 进行转置，即 `a.transpose(0, 1)` 或 `np.transpose(a, (1, 0, 2))`

3. 转置 (0, 1, 2) -> (2, 1, 0)

- 操作：交换“格”和“列”。
- 效果：这是最彻底的“旋转”。原来的第几格，变成了现在的第几列；而“行”保持不变。
 - 原来第0格第1行第2列的那本书，现在位于第2格第1行第0列。
 - 想象一下：你把书架侧着旋转了90度，从侧面看过去。

2.1 标量、向量、矩阵和张量

19

- import torch
- import numpy as np
- example = torch.tensor(np.arange(24).reshape([2, 3, 4]))

这个三阶张量是由2个3行4列的矩阵构成

```
tensor([[[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]],
        [[12, 13, 14, 15],
         [16, 17, 18, 19],
         [20, 21, 22, 23]]], dtype=torch.int32)
```

example2 = example.transpose(0, 1)

```
tensor([[[ 0,  1,  2,  3],
         [12, 13, 14, 15]],
        [[ 4,  5,  6,  7],
         [16, 17, 18, 19]],
        [[ 8,  9, 10, 11],
         [20, 21, 22, 23]]], dtype=torch.int32)
```

2.1 标量、向量、矩阵和张量

20

example2 = example.transpose(0, 2)

```
tensor([[[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]],

        [[12, 13, 14, 15],
          [16, 17, 18, 19],
          [20, 21, 22, 23]]], dtype=torch.int32)
```

```
tensor([[[ 0, 12],
          [ 4, 16],
          [ 8, 20]],

        [[ 1, 13],
          [ 5, 17],
          [ 9, 21]],

        [[ 2, 14],
          [ 6, 18],
          [10, 22]],

        [[ 3, 15],
          [ 7, 19],
          [11, 23]]], dtype=torch.int32)
```

2.1 标量、向量、矩阵和张量

21

```
import torch
import numpy as np
from PIL import Image
example = torch.tensor(np.arange(180).reshape([3, 20, 3]))
x=example
y=example.transpose(0, 2)
x = x.numpy()
y=y.numpy()
x = x.astype(np.uint8)
y = y.astype(np.uint8)
imagex = Image.fromarray(x)
imagey = Image.fromarray(y)
imagex.show()
imagey.show()
```

x的图像



y的图像



2.1 标量、向量、矩阵和张量

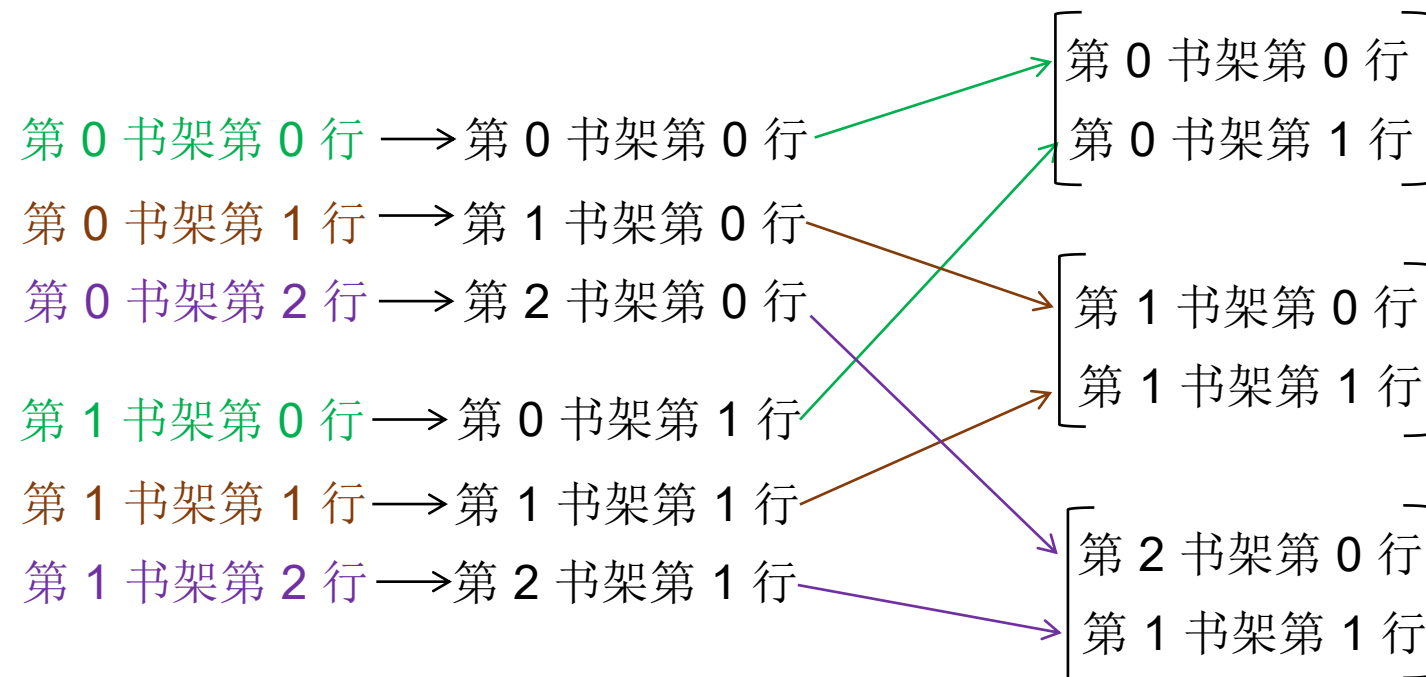
22

练习题

```
import torch
import numpy as np
example = torch.tensor(np.arange(18).reshape([2, 3, 3]))
example1 = example.transpose(0, 1)
example2 = example.transpose(0, 2)
```

分别画出或写出example、example1、example2


```
import numpy as np
import torch
example = torch.tensor(np.arange(18).reshape(2, 3, 3))
example1 = example.transpose(dim0: 0, dim1: 1)
print(example)
print("#####")
print(example1)
```



```
D:\Ksoftware\miniconda3\envs\eventgp38\python.e
tensor([[[ 0,  1,  2],
          [ 3,  4,  5],
          [ 6,  7,  8]],

        [[ 9, 10, 11],
          [12, 13, 14],
          [15, 16, 17]]], dtype=torch.int32)

#####
tensor([[[ 0,  1,  2],
          [ 9, 10, 11]],

        [[ 3,  4,  5],
          [12, 13, 14]],

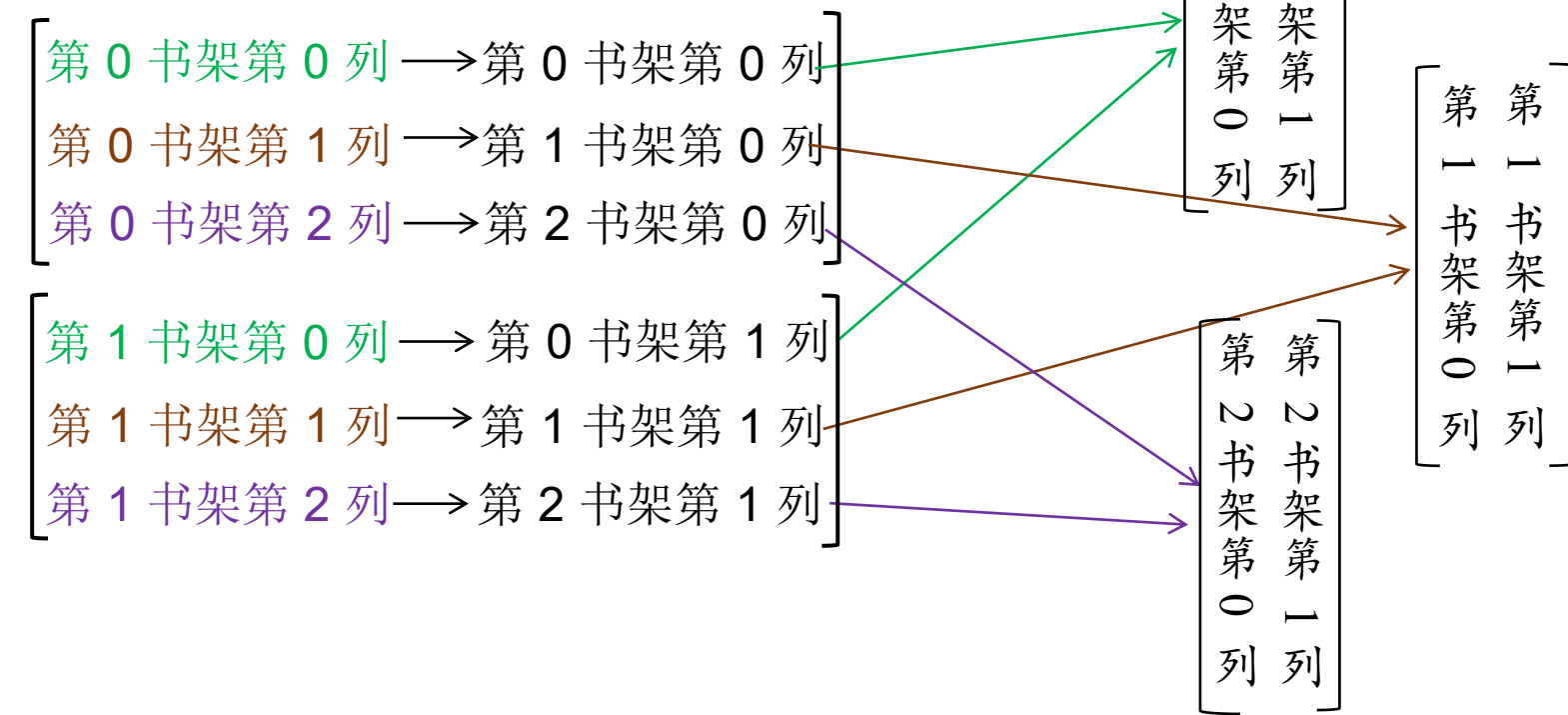
        [[ 6,  7,  8],
          [15, 16, 17]]], dtype=torch.int32)

Process finished with exit code 0
```

```
import numpy as np
import torch
example = torch.tensor(np.arange(18).reshape(2, 3, 3))
example2 = example.transpose(dim0: 0, dim1: 2)
print(example)
print("#####")
print(example2)
```

转置 (0, 1, 2) -> (2, 1, 0)

原来的第几格，变成了现在的第几列；而“行”保持不变



```
tensor([[[ 0,  1,  2],
          [ 3,  4,  5],
          [ 6,  7,  8]],

        [[ 9, 10, 11],
          [12, 13, 14],
          [15, 16, 17]]], dtype=torch.int32)

#####
tensor([[[ 0,  9],
          [ 3, 12],
          [ 6, 15]],

        [[ 1, 10],
          [ 4, 13],
          [ 7, 16]],

        [[ 2, 11],
          [ 5, 14],
          [ 8, 17]]], dtype=torch.int32)
```


2.1 标量、向量、矩阵和张量

23

只要矩阵的形状一样，我们可以把两个矩阵相加。两个矩阵相加是指对应位置的元素相加，比如 $\mathbf{C} = \mathbf{A} + \mathbf{B}$ ，其中 $C_{i,j} = A_{i,j} + B_{i,j}$ 。

标量和矩阵相乘，或是和矩阵相加时，我们只需将其与矩阵的每个元素相乘或相加，比如 $\mathbf{D} = a \cdot \mathbf{B} + c$ ，其中 $D_{i,j} = a \cdot B_{i,j} + c$ 。

2.1 标量、向量、矩阵和张量

24

在深度学习中，我们也使用一些不那么常规的符号。我们允许矩阵和向量相加，产生另一个矩阵： $C = A + b$ ，其中 $C_{i,j} = A_{i,j} + b_j$ 。换言之，向量 b 和矩阵 A 的每一行相加。这个简写方法使我们无需在加法操作前定义一个将向量 b 复制到每一行而生成的矩阵。这种隐式地复制向量 b 到很多位置的方式，被称为广播 (broadcasting)。

2.1 标量、向量、矩阵和张量

25

如果相加的两个数组的shape不同, 就会触发广播机制

- 1) 程序会自动执行操作使得 $A.shape == B.shape$;
- 2) 对应位置进行相加运算, 结果的shape是对应位置的最大值

2.1 标量、向量、矩阵和张量

26

能够广播的条件:

- 每个tensor至少有一个维度;
- 遍历tensor所有维度时, 从末尾开始遍历 (从右往左开始遍历) (从后往前开始遍历), 两个tensor存在下列情况:
 - tensor维度相等。
 - tensor维度不等且其中一个维度为1。
 - tensor维度不等且其中一个维度不存在。

或

```
x=torch.ones((0,))  
y=torch.ones(5,7,3)  
z = x+y
```

```
compute_result.py x
1 import numpy as np
2 import torch
3
4 x=torch.ones((0,))
5 print(x.shape)
6 print(x)
7 y=torch.ones(5,7,3)
8 z = x+y
9
10 print(z.shape)
```

compute_result x

D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myw
torch.Size([0])
tensor([])
Traceback (most recent call last):
File "D:\myworks\event-as-GP\data_pretreatment\compute
z = x+y
RuntimeError: The size of tensor a (0) must match the s
Process finished with exit code 1

2.1 标量、向量、矩阵和张量

27

```
x = torch.ones(5,3,4, 1)
y = torch.ones(3, 1, 1)
z = x+y
```

如何广播？

z的shape是什么？

z的每个元素是什么？

从尾部维度开始遍历

- tensor维度相等。
- tensor维度不等且其中一个维度为1。
- tensor维度不等且其中一个维度不存在。

2.1 标量、向量、矩阵和张量

28

```
x = torch.ones(5,2,4, 1)
y = torch.ones(3, 1, 1)
z = x+y
```

如何广播?

z的shape是什么?

z的每个元素是什么?

```
compute_result.py x
1  import numpy as np
2  import torch
3
4  x = torch.ones(5,3,4, 1)
5  #print(x)
6  y = torch.ones(3, 1, 1)
7  print(y)
8  z = x+y
9  #print(z)
10 print(z.shape)
11

compute_result x
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\eventgp38\compute_result.py
tensor([[[[1.]],
          [[1.]],
          [[1.]]],
        [[1.]],
        [[1.]]])
torch.Size([5, 3, 4, 1])

Process finished with exit code 0
```

2.1 标量、向量、矩阵和张量

29

```
x = torch.ones(5,2,4, 1)
y = torch.ones(1, 1)
z = x+y
```

如何广播?

z的shape是什么?

z的每个元素是什么?

```
1 import numpy as np
2 import torch
3
4 x = torch.ones(5,2,4, 1)
5 y = torch.ones(1, 1)
6 z = x+y
7 print(z)
8 print(z.shape)
9
```

compute_result x

```
[[2.],
 [2.],
 [2.],
 [2.]],

[[[2.],
 [2.],
 [2.],
 [2.]],

 [[2.],
 [2.],
 [2.],
 [2.]],

 [[[2.],
 [2.],
 [2.],
 [2.]],

 [[2.],
 [2.],
 [2.],
 [2.]]]]
torch.Size([5, 2, 4, 1])

Process finished with exit code 0
```

2.2 矩阵和向量相乘

30

矩阵乘法是矩阵运算中最重要的操作之一。两个矩阵 A 和 B 的矩阵乘积 (matrix product) 是第三个矩阵 C 。为了使乘法定义良好，矩阵 A 的列数必须和矩阵 B 的行数相等。如果矩阵 A 的形状是 $m \times n$ ，矩阵 B 的形状是 $n \times p$ ，那么矩阵 C 的形状是 $m \times p$ 。我们可以通过将两个或多个矩阵并列放置以书写矩阵乘法，例如

$$C = AB. \tag{2.4}$$

具体地，该乘法操作定义为

$$C_{i,j} = ? \tag{2.5}$$

2.2 矩阵和向量相乘

31

矩阵乘法是矩阵运算中最重要的操作之一。两个矩阵 A 和 B 的矩阵乘积 (matrix product) 是第三个矩阵 C 。为了使乘法定义良好，矩阵 A 的列数必须和矩阵 B 的行数相等。如果矩阵 A 的形状是 $m \times n$ ，矩阵 B 的形状是 $n \times p$ ，那么矩阵 C 的形状是 $m \times p$ 。我们可以通过将两个或多个矩阵并列放置以书写矩阵乘法，例如

$$C = AB. \tag{2.4}$$

具体地，该乘法操作定义为

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}. \tag{2.5}$$

2.2 矩阵和向量相乘

32

需要注意的是，两个矩阵的标准乘积不是指两个矩阵中对应元素的乘积。不过，那样的矩阵操作确实是存在的，被称为 **元素对应乘积**（element-wise product）或者 **Hadamard 乘积**（Hadamard product），记为 $A \odot B$ 。

2.2 矩阵和向量相乘

33

两个相同维数的向量 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 的点积 (dot product) 可看作是矩阵乘积 $\boldsymbol{x}^\top \boldsymbol{y}$ 。我们可以把矩阵乘积 $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$ 中计算 $C_{i,j}$ 的步骤看作是 $\boldsymbol{A}$ 的第 i 行和 $\boldsymbol{B}$ 的第 j 列之间的点积。

2.2 矩阵和向量相乘

34

矩阵乘积运算有许多有用的性质，从而使矩阵的数学分析更加方便。比如，矩阵乘积服从分配律：

$$A(B + C) = AB + AC. \quad (2.6)$$

矩阵乘积也服从结合律：

$$A(BC) = (AB)C. \quad (2.7)$$

2.2 矩阵和向量相乘

35

不同于标量乘积，矩阵乘积并不满足交换律（ $AB = BA$ 的情况并非总是满足）。然而，两个向量的点积（dot product）满足交换律：

$$\mathbf{x}^\top \mathbf{y} = \mathbf{y}^\top \mathbf{x}. \quad (2.8)$$

矩阵乘积的转置有着简单的形式：

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top. \quad (2.9)$$

利用两个向量点积的结果是标量，标量转置是自身的事实，我们可以证明式(2.8)：

$$\mathbf{x}^\top \mathbf{y} = (\mathbf{x}^\top \mathbf{y})^\top = \mathbf{y}^\top \mathbf{x}. \quad (2.10)$$

2.2 矩阵和向量相乘

36

- `>>> a = torch.tensor([[1, 2], [2, 3]])`
- `>>> a * a`

```
compute_result.py x
1 import numpy as np
2 import torch
3
4 a = torch.tensor([[1, 2], [2, 3]])
5 print(a)
6 b = a*a
7 print("*****")
8 print(b.shape)
9 print("oooooooooooooooooooo")
10 print(b)

Run compute_result x
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\m
tensor([[1, 2],
        [2, 3]])
*****
torch.Size([2, 2])
oooooooooooooooooooo
tensor([[1, 4],
        [4, 9]])

Process finished with exit code 0
```

2.2 矩阵和向量相乘

37

- `torch.mul`

实现的是**按元素乘**。如果矩阵a, b的维度不同, 会按照**广播机制**先分别将a, b的数据进行扩充, 如果能够扩充到同维度, 再进行元素乘。

- `torch.mm`

实现**矩阵乘法**。A的维度如果是 $m \times n$, B的维度如果是 $n \times p$, 则`torch.mm(A,B)`的维度是 $m \times p$ 。此处A, B**只能是二维的张量**

- `torch.bmm`

按**批次**的**矩阵乘法**。因为深度学习中大多数时候是一个batch_size一个batch_size的读入数据, pytorch对此也进行了实现, 即`torch.bmm`。A的维度是 $b \times m \times n$, B的维度是 $b \times n \times p$, 则`torch.bmm(A,B)`的维度是 $b \times m \times p$ 。此处A, B**只能是三维的张量**。

- `torch.dot`

点积。A是维度m的向量, B是维度m的向量。 `torch.dot(A,B)`返回按元素乘后的求和结果, 即点积结果。注意此处要求A, B均为1D的张量。

2.2 矩阵和向量相乘

38

- torch.mul

实现的是**按元素乘**。如果矩阵a, b的维度不同, 会按照广播机制先分别将a, b的数据进行扩充, 如果能够扩充到同维度, 再进行元素乘。

```
import torch
a = torch.ones(3,4)
b = torch.Tensor([1,2,3]).reshape((3, 1))
c=torch.mul(a, b)
print(a)
print(b)
print(c)
```

2.2 矩阵和向量相乘

39

- torch.mul

实现的是**按元素乘**。如果矩阵a, b的维度不同, 会按照**广播机制**先分别将a, b的数据进行扩充, 如果能够扩充到同维度, 再进行元素乘。

```
import torch
a = torch.ones(3,4)
print(a)
b = torch.Tensor([1,2,3]).reshape((3, 1))
print(b)
print(torch.mul(a, b))
```

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
tensor([[1.],
        [2.],
        [3.]])
tensor([[1., 1., 1., 1.],
        [2., 2., 2., 2.],
        [3., 3., 3., 3.]])
```

2.2 矩阵和向量相乘

40

- torch.mm

实现**矩阵乘法**。A的维度如果是 $m \times n$ ，B的维度如果是 $n \times p$ ，则torch.mm(A,B)的维度是 $m \times p$ 。此处A，B只能是二维的张量

```
mat1 = torch.arange(12).reshape(3,4)
```

```
mat2 = torch.arange(12).reshape(4,3)
```

```
torch.mm(mat1, mat2)
```

compute_result.py x

```
1 import numpy as np
2 import torch
3
4 mat1 = torch.arange(12).reshape(*shape: 3, 4)
5 mat2 = torch.arange(12).reshape(*shape: 4, 3)
6 mat3 = torch.mm(mat1, mat2)
7 print(mat1)
8 print("oooooooooooooooooooooooooooo")
9 print(mat2)
10 print("#####")
11 print(mat3)
12
13
```

un compute_result x

```
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
oooooooooooooooooooooooooooo
tensor([[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8],
        [ 9, 10, 11]])
#####
tensor([[ 42,  48,  54],
        [114, 136, 158],
        [186, 224, 262]])
```

Process finished with exit code 0

2.2 矩阵和向量相乘

41

- torch.bmm

按批次的矩阵乘法。因为深度学习中大多数时候是一个batch_size的读入数据，pytorch对此也进行了实现，即torch.bmm。A的维度是 $b*m*n$ ，B的维度是 $b*n*p$ ，则torch.bmm(A,B)的维度是 $b*m*p$ 。此处A，B只能是三阶的张量。

- `a=torch.Tensor([[[3,4],[1,2]],
 [[3,4],[1,2]],
 [[3,4],[1,2]])`
- `b=torch.Tensor([[[1,2],[3,4]],`
- `[[1,2],[3,4]],`
- `[[3,4],[ 1,2]])`

```
Out[50]: tensor([[[15., 22.],  
                  [ 7., 10.]],  
                [[15., 22.],  
                  [ 7., 10.]],  
                [[13., 20.],  
                  [ 5.,  8.]])
```

`torch.bmm(a,b)`

2.2 矩阵和向量相乘

42

- torch.dot

点积。A是维度m的向量， B是维度m的向量。 torch.dot(A,B)返回按元素乘后的求和结果， 即点积结果。 注意此处要求A， B均为1D的张量。

```
a = torch.tensor([2, 3])  
b = torch.tensor([2, 1])  
c = torch.dot(a,b)  
print(c)
```

```
1  import numpy as np  
2  import torch  
3  
4  a = torch.tensor([2, 3])  
5  b = torch.tensor([2, 1])  
6  c = torch.dot(a,b)  
7  print(c)  
8  
9
```

compute_result x

⋮

D:\Ksoftware\miniconda3\envs\eventgp38\python
tensor(7)

Process finished with exit code 0

2.2 矩阵和向量相乘

43

`torch.matmul(input, other, *, out=None)`

- 如果两个张量都是一维的，则返回点积（标量）。`torch.dot`
- 如果两个参数都是二维的，则返回矩阵-矩阵乘积。`torch.mm`
- 如果第一个参数是一维的，第二个参数是二维的，为了矩阵乘法的目的，在它的维数前面加上一个 1。在矩阵相乘之后，前置维度被移除。
- 如果第一个参数是二维的，第二个参数是一维的，则返回矩阵向量积。`torch.mul`
- 如果两个参数至少为一维且至少一个参数为 N 维（其中 $N > 2$ ），则返回批处理矩阵乘法 **区别于 `torch.bmm`**
 - 如果第一个参数是一维的，则将 1 添加到其维度，以便批量矩阵相乘并在之后删除。如果第二个参数是一维的，则将 1 附加到其维度以用于批量矩阵倍数并在之后删除
 - 非矩阵（即批次）维度是广播的（因此必须是可广播的）**广播是从末尾开始看**
 - 例如，如果输入是 $(j \times 1 \times n \times n)$ 张量
 - 另一个是 $(k \times n \times n)$ 张量，
 - `out` 将是一个 $(j \times k \times n \times n)$ 张量

2.2 矩阵和向量相乘

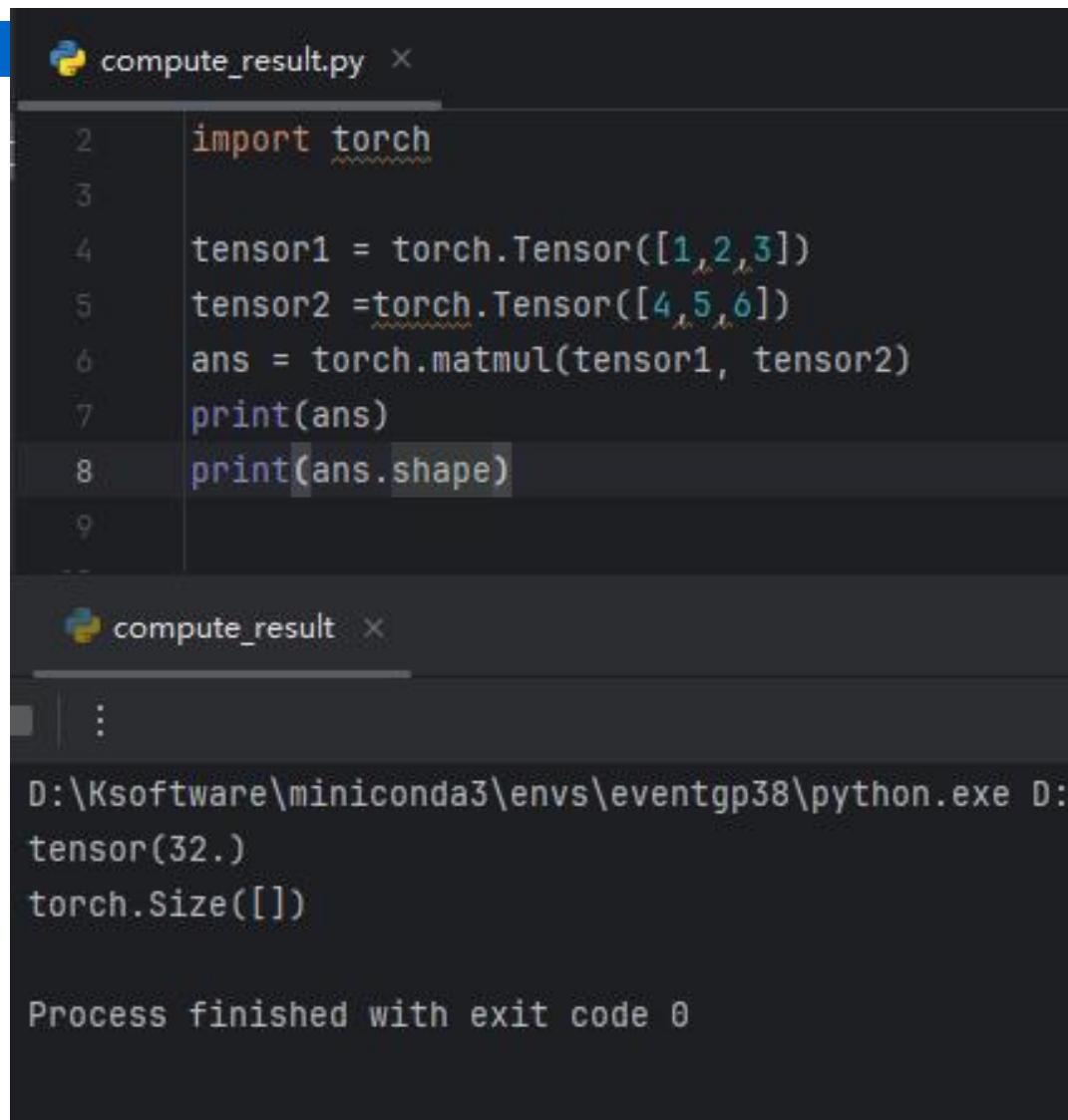
44

`torch.matmul(input, other, *, out=None)`

`tensor1 = torch.Tensor([1,2,3])`

`tensor2 = torch.Tensor([4,5,6])`

`ans = torch.matmul(tensor1, tensor2)`



The screenshot shows a Python IDE with two windows. The top window, titled 'compute_result.py', contains the following code:

```
1
2 import torch
3
4 tensor1 = torch.Tensor([1,2,3])
5 tensor2 = torch.Tensor([4,5,6])
6 ans = torch.matmul(tensor1, tensor2)
7 print(ans)
8 print(ans.shape)
9
```

The bottom window, titled 'compute_result', shows the output of the script:

```
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:
tensor(32.)
torch.Size([1])

Process finished with exit code 0
```

2.2 矩阵和向量相乘

45

`torch.matmul(input, other, *, out=None)`

```
tensor1 = torch.Tensor([[1,2,3],[1,2,3]])
```

```
tensor2 = torch.Tensor([[4,5],[4,5],[4,5]])
```

```
ans = torch.matmul(tensor1, tensor2)
```

```
compute_result.py x
5 print(tensor1.shape)
6 tensor2 = torch.Tensor([[4,5],[4,5],[4,5]])
7 print(tensor2.shape)
8 ans = torch.matmul(tensor1, tensor2)
9 print("#####")
10 print(ans)
11 print(ans.shape)
12

compute_result x
:
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\mywor
torch.Size([2, 3])
torch.Size([3, 2])
#####
tensor([[24., 30.],
        [24., 30.]])
torch.Size([2, 2])

Process finished with exit code 0
```

2.2 矩阵和向量相乘

46

`torch.matmul(input, other, *, out=None)`

`tensor1 = torch.Tensor([1,2,3])` **# 注意这里是一维**

`tensor2 = torch.Tensor([[4,5],[4,5],[4,5]])`

`ans = torch.matmul(tensor1, tensor2)`

如果第一个参数是一维的，第二个参数是二维的，为了矩阵的乘法的目的，在它的维数前面加上一个1，在矩阵相乘之后，前置维度被移除。

`[1,2,3]` $\longrightarrow$ `[[1,2,3]]`

`torch.matmul([[1,2,3]], [[4,5],[4,5],[4,5]])`

```
compute_result.py x
1  import numpy as np
2  import torch
3
4  tensor1 = torch.Tensor([1,2,3]) # 注意这里是一维
5  print(tensor1.shape)
6  tensor2 = torch.Tensor([[4,5],[4,5],[4,5]])
7  print(tensor2.shape)
8  ans = torch.matmul(tensor1, tensor2)
9  print("#####")
10 print(ans)
11 print(ans.shape)
12

compute_result x
:
D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks
torch.Size([3])
torch.Size([3, 2])
#####
tensor([24., 30.])
torch.Size([2])

Process finished with exit code 0
```


2.2 矩阵和向量相乘

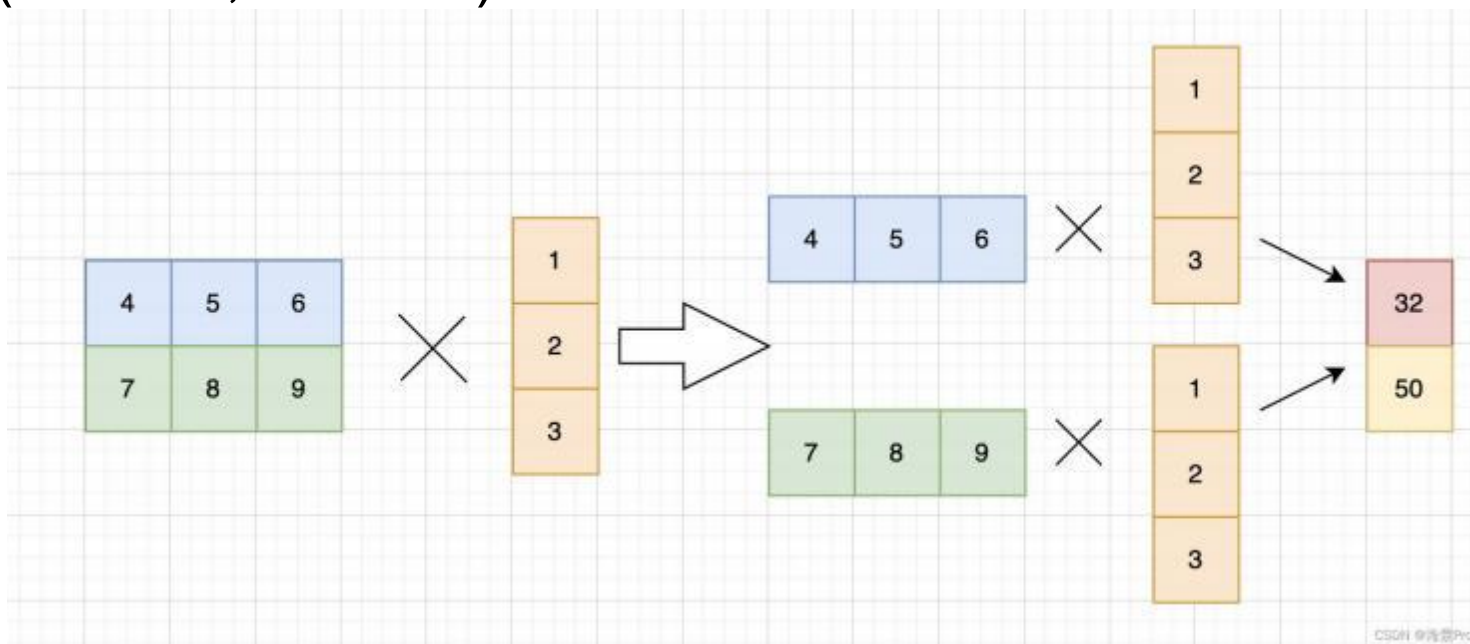
47

```
torch.matmul(input, other, *, out=None)
```

```
tensor1 = torch.Tensor([[4,5,6],[7,8,9]])
```

```
tensor2 = torch.Tensor([1,2,3])
```

```
ans = torch.matmul(tensor1, tensor2)
```



2.2 矩阵和向量相乘

48

```
torch.matmul(input, other, *, out=None)
```

执行批量矩阵向量乘法

```
tensor1 = torch.randn(10, 3, 4)
```

```
tensor2 = torch.randn(4)
```

```
print(torch.matmul(tensor1, tensor2).shape)
```

```
1 import numpy as np
2 import torch
3
4 tensor1 = torch.randn(10, 3, 4)
5 print(tensor1.shape)
6 tensor2 = torch.randn(4)
7 print(tensor2.shape)
8 ans = torch.matmul(tensor1, tensor2)
9 print("#####")
10 print(ans.shape)
11
```

compute_result x

:

D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\mywork
torch.Size([10, 3, 4])
torch.Size([4])

torch.Size([10, 3])

Process finished with exit code 0

运算规则解读

当 `tensor1` 是一个3D张量（形状为 `(10, 3, 4)`），而 `tensor2` 是一个1D张量（形状为 `(4,)`）时，`torch.matmul` 的操作逻辑如下：

- **批量矩阵乘法**：将 `tensor1` 视为一个包含10个独立矩阵的批次，每个矩阵的形状是 `(3, 4)`。
- **向量维度扩充**：将1D的 `tensor2`（形状 `(4,)`）临时视为一个形状为 `(4, 1)` 的矩阵（相当于一个列向量），以进行矩阵乘法⁸。
- **执行乘法**：每个 `(3, 4)` 的矩阵与临时列向量 `(4, 1)` 相乘。根据矩阵乘法规则 `(3,4) @ (4,1) -> (3,1)`，结果会得到10个形状为 `(3, 1)` 的矩阵。
- **压缩尾部维度**：最后，函数会移除结果中每个矩阵末尾那个为1的维度⁸。因此，10个 `(3, 1)` 的矩阵就变成了10个长度为3的向量，最终输出形状为 `(10, 3)`。

简单来说，这个过程就像是 `tensor1` 中的每一个 `(3,4)` 小矩阵，都分别与 `tensor2` 这个向量做了一次矩阵乘法。

2.2 矩阵和向量相乘

48

```
tensor1 = torch.randn(10, 2, 3, 4) #  
tensor2 = torch.randn(4)  
print(torch.matmul(tensor1, tensor2).shape)
```

执行批量矩阵向量乘法

```

1 import numpy as np
2 import torch
3
4 tensor1 = torch.randn(10, 2, 3, 4) #
5 print(tensor1.shape)
6 tensor2 = torch.randn(4)
7 print(tensor2.shape)
8 ans = torch.matmul(tensor1, tensor2)
9 print("#####")
10 print(ans.shape)

```

compute_result x

```

D:\Ksoftware\miniconda3\envs\eventgp38\python.exe D:\myworks\event
torch.Size([10, 2, 3, 4])
torch.Size([4])
#####
torch.Size([10, 2, 3])

```

Process finished with exit code 0

运算规则解读

`torch.matmul` 在处理像你例子中这样的高维张量和向量时，遵循以下规则⁷：

- **批量矩阵乘法**：函数会忽略最前面的维度（在这个例子中是 10 和 2），将它们视为**批次维度**。运算会在这两个批次维度下进行。
- **向量维度扩充**：对于 1 维的 `tensor2`（形状 (4,)），`torch.matmul` 会自动在其末尾添加一个维度，将其临时视为一个形状为 (4, 1) 的列向量，以便进行矩阵乘法⁷¹⁰。
- **执行乘法**：对于 `tensor1` 中每一个具体的 (3, 4) 矩阵，都与临时变成 (4, 1) 列向量的 `tensor2` 进行矩阵乘法。根据矩阵乘法规则 $(3, 4) @ (4, 1) \rightarrow (3, 1)$ ，结果是一个 (3, 1) 的矩阵。
- **压缩尾部维度**：最后，函数会移除结果中每个矩阵末尾那个为 1 的维度（也就是“压缩”掉那个维度）⁷¹⁰。于是，(3, 1) 就变成了 (3,)。所有 $10 * 2$ 个这样的结果组合起来，就得到了 (10, 2, 3) 形状的输出。

简单来说，这个过程就像是 `tensor1` 中的每一个 (3,4) 小矩阵，都分别与 `tensor2` 这个向量做了一次矩阵乘法。

2.2 矩阵和向量相乘

49

```
torch.matmul(input, other, *, out=None)
tensor1 = torch.randn(4)
tensor2 = torch.randn(10, 4, 3)
print(torch.matmul(tensor1, tensor2).size())
```

```
tensor1 = torch.randn(4)
tensor2 = torch.randn(10, 2, 4, 3)
print(torch.matmul(tensor1, tensor2).size())
```

```
>>> tensor1 = torch.randn(10, 3, 4)
>>> tensor2 = torch.randn(10, 4, 5)
>>> torch.matmul(tensor1, tensor2).size()
torch.Size([10, 3, 5])
```



2.2 矩阵和向量相乘

50

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad (2.11)$$

其中 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 是一个已知矩阵， $\mathbf{b} \in \mathbb{R}^m$ 是一个已知向量， $\mathbf{x} \in \mathbb{R}^n$ 是一个我们要求解的未知向量。向量 $\mathbf{x}$ 的每一个元素 x_i 都是未知的。矩阵 $\mathbf{A}$ 的每一行和 $\mathbf{b}$ 中对应的元素构成一个约束。我们可以把式 (2.11) 重写为

$$\begin{array}{ccc} \mathbf{A}_{1,:}\mathbf{x} = b_1 & & \mathbf{A}_{1,1}x_1 + \mathbf{A}_{1,2}x_2 + \cdots \mathbf{A}_{1,n}x_n = b_1 \\ \mathbf{A}_{2,:}\mathbf{x} = b_2 & \Rightarrow & \mathbf{A}_{2,1}x_1 + \mathbf{A}_{2,2}x_2 + \cdots \mathbf{A}_{2,n}x_n = b_2 \\ \dots & & \dots \\ \mathbf{A}_{m,:}\mathbf{x} = b_m & & \mathbf{A}_{m,1}x_1 + \mathbf{A}_{m,2}x_2 + \cdots \mathbf{A}_{m,n}x_n = b_m. \end{array}$$

2.5 范数

51

有时我们需要衡量一个向量的大小。在机器学习中，我们经常使用被称为 **范数** (norm) 的函数衡量向量大小。形式上， L^p 范数定义如下

$$\|\mathbf{x}\|_p = \left(\sum_i |x_i|^p \right)^{\frac{1}{p}} \quad (2.30)$$

其中 $p \in \mathbb{R}$, $p \geq 1$ 。

2.5 范数

52

当 $p = 2$ 时, L^2 范数被称为 欧几里得范数 (Euclidean norm)。它表示从原点出发到向量 $\mathbf{x}$ 确定的点的欧几里得距离。 L^2 范数在机器学习中出现地十分频繁, 经常简化表示为 $\|\mathbf{x}\|$, 略去了下标 2。平方 L^2 范数也经常用来衡量向量的大小, 可以简单地通过点积 $\mathbf{x}^\top \mathbf{x}$ 计算。

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

2.5 范数

53

平方 L^2 范数在数学和计算上都比 L^2 范数本身更方便。例如，平方 L^2 范数对 $\mathbf{x}$ 中每个元素的导数只取决于对应的元素，而 L^2 范数对每个元素的导数却和整个向量相关。但是在很多情况下，平方 L^2 范数也可能不受欢迎，因为它在 origin 附近增长得十分缓慢。在某些机器学习应用中，区分恰好是零的元素和非零但值很小的元素是很重要的。在这些情况下，我们转而使用在各个位置斜率相同，同时保持简单的数学形式的函数： L^1 范数。 L^1 范数可以简化如下：

$$\|\mathbf{x}\|_1 = \sum_i |x_i|. \quad (2.31)$$

当机器学习问题中零和非零元素之间的差异非常重要时，通常会使用 L^1 范数。每当 $\mathbf{x}$ 中某个元素从 0 增加 ϵ ，对应的 L^1 范数也会增加 ϵ 。

2.5 范数

54

另外一个经常在机器学习出现的范数是 L^∞ 范数，也被称为 **最大范数** (max norm)。这个范数表示向量中具有最大幅值的元素的绝对值：

$$\|\mathbf{x}\|_\infty = \max_i |x_i|. \quad (2.32)$$

2.5 范数

55

两个向量的点积（dot product）可以用范数来表示。具体地，

$$\mathbf{x}^\top \mathbf{y} = \|\mathbf{x}\|_2 \|\mathbf{y}\|_2 \cos \theta \quad (2.34)$$

其中 θ 表示 $\mathbf{x}$ 和 $\mathbf{y}$ 之间的夹角。

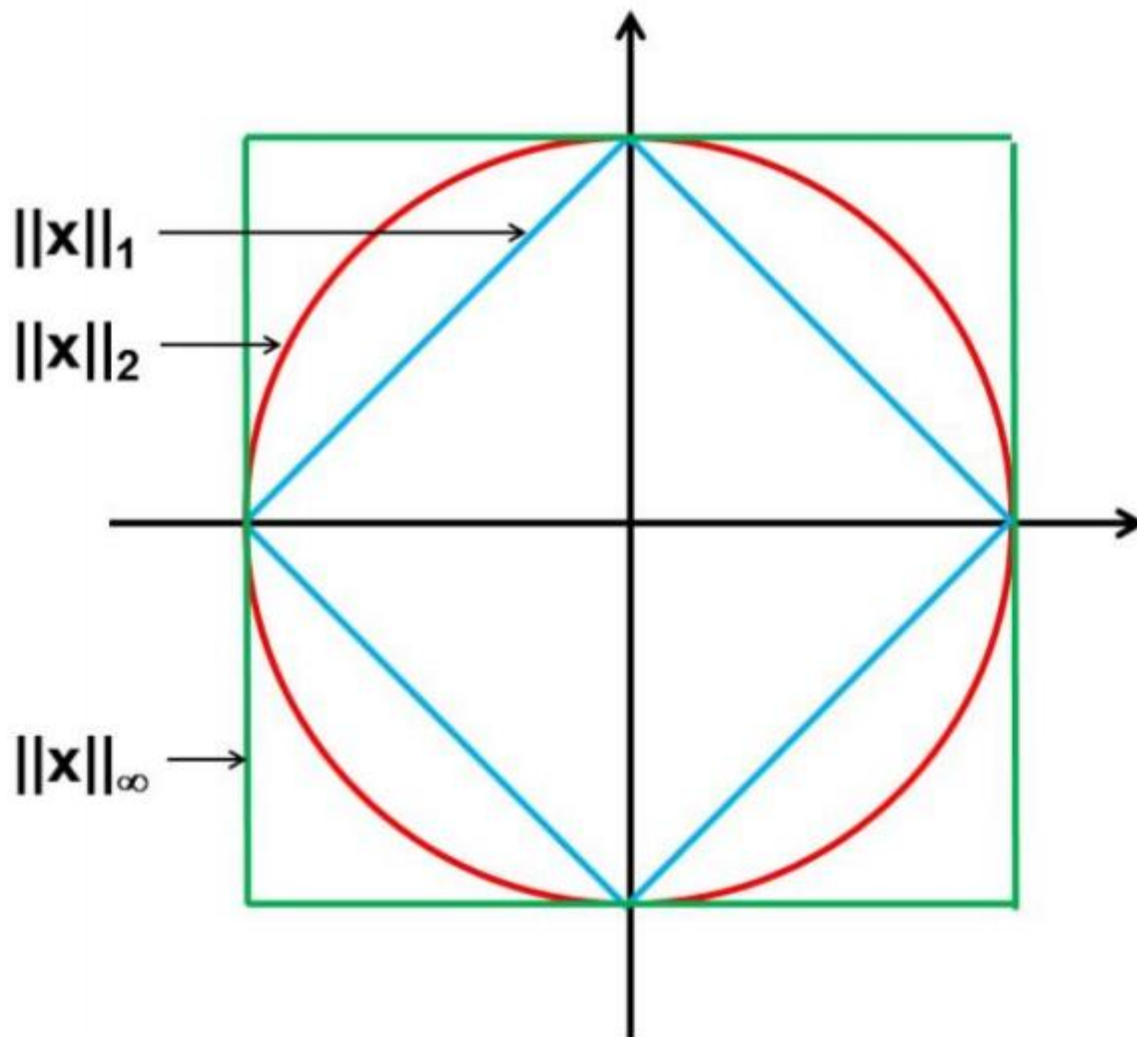
2.5 范数

56

$$\|\mathbf{x}\|_1 = \sum_i |x_i|.$$

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

$$\|\mathbf{A}\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$$



2.5 范数

57

5. 向量 $X=[1,2,3,-4,9,0]$ 的 $L1$ 范数为 ()

A. 11

B. 19

C. 6

D. $\sqrt{(111)}$

L^1 范数可以简化如下:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|.$$

练习题

58

```
>>> e = eye(3)
>>> e
array([[ 1.,  0.,  0.],
       [ 0.,  1.,  0.],
       [ 0.,  0.,  1.]])
>>> e.shape
```

练习题

59

```
>>> b =array([1,2,3,4])
```

练习题

60

```
>>> c = array([[1, 1],[ 1,2],[ 1,3],[ 1,4]])
```

```
>>> c.shape
```

```
>>> shape(3)
```

练习题

61

```
>>> import torch
>>> A=torch.ones(2,3)
>>> B=2*torch.ones(4,3)
>>> C=torch.cat((A,B),0)

>>> C.size()

>>> D=2*torch.ones(2,4) #2x4的张量 (矩阵)
>>> C=torch.cat((A,D), 1)
```

练习题

62

```
a = torch.ones(3,4)
b = torch.ones(4,2)
torch.mm(a, b)
```

练习题

63

```
a = torch.ones(3,4)
b = torch.ones(5,4,2)
torch.matmul(a, b).shape
```

练习题

64

```
a = torch.ones(5,4,2)
b = torch.ones(5,2,3)
torch.matmul(a, b).shape
```

```
a = torch.ones(5,4,2)
b = torch.ones(5,2,3)
torch.matmul(b, a).shape
```


2.2 矩阵和向量相乘

65

`torch.matmul(input, other, *, out=None)`

- 如果两个张量都是一维的，则返回点积（标量）。
- 如果两个参数都是二维的，则返回矩阵-矩阵乘积。
- 如果第一个参数是一维的，第二个参数是二维的，为了矩阵乘法的目的，在它的维数前面加上一个 1。在矩阵相乘之后，前置维度被移除。
- 如果第一个参数是二维的，第二个参数是一维的，则返回矩阵向量积。
- 如果两个参数至少为一维且至少一个参数为 N 维（其中 $N > 2$ ），则返回批处理矩阵乘法
 - 如果第一个参数是一维的，则将 1 添加到其维度，以便批量矩阵相乘并在之后删除。如果第二个参数是一维的，则将 1 附加到其维度以用于批量矩阵倍数并在之后删除
 - 非矩阵（即批次）维度是广播的（因此必须是可广播的）
 - 例如，如果输入是 $(j \times 1 \times n \times n)$ 张量
 - 另一个是 $(k \times n \times n)$ 张量，
 - `out` 将是一个 $(j \times k \times n \times n)$ 张量